



Grado en Ingeniería Informática

Anteproyecto TFG

Datos del proyecto

Asistente Contextual para Gestión de Conocimiento Interno
Título

Datos del estudiante

Gardón Martín, Gonzalo
Apellidos, Nombre

ggardonma.inf@upsa.es
Email

722279940
Teléfono

C/ Carnaval del Toro 20, Ciudad Rodrigo 37500, Salamanca, España
Dirección

Datos del director/co-director

Roberto Berjón Gallinas
Nombre director

Nombre co-director

Presentación

El Trabajo de Fin de Grado plantea el desarrollo de una aplicación backend basada en Quarkus que integre las capacidades de los Modelos de Lenguaje Extensos (LLMs) mediante la biblioteca LangChain4j. El proyecto consiste en construir un asistente virtual capaz de responder preguntas a partir de información almacenada en bases de datos o en documentos (PDF, Word, etc.) utilizando técnicas de Retrieval-Augmented Generation (RAG).

Además, el asistente podrá ejecutar acciones dentro del sistema mediante el estándar Model Context Protocol (MCP), permitiendo una interacción dinámica con el backend. La comunicación con el usuario se realizará a través de WebSockets, proporcionando una experiencia fluida y en tiempo real.

Este TFG combina tecnologías actuales en el ámbito de la inteligencia artificial aplicada al desarrollo backend, junto con patrones modernos para la integración de LLMs en aplicaciones empresariales.

Este proyecto surge del interés por explorar cómo los LLMs pueden integrarse en arquitecturas backend modernas y aportar valor en la gestión de conocimiento en entornos empresariales.



Objetivos

Objetivo general

Desarrollar un asistente virtual integrado en una aplicación basada en Quarkus y LangChain4j, capaz de consultar información contextual mediante técnicas de Retrieval-Augmented Generation (RAG) y de ejecutar acciones en el backend a través del estándar Model Context Protocol (MCP).

Objetivos específicos

1. Analizar el estado actual de las tecnologías relacionadas con LLMs, RAG y el framework LangChain4j.
2. Diseñar una arquitectura de backend en Quarkus que permita la integración y comunicación con modelos de lenguaje.
3. Implementar un sistema de recuperación de información basado en embeddings y almacenamiento vectorial.
4. Integrar el estándar MCP para definir y exponer herramientas que permitan al modelo ejecutar acciones dentro del sistema.
5. Desarrollar un módulo de comunicación en tiempo real mediante WebSockets para la interacción con el asistente.
6. Evaluar la precisión y eficacia del asistente al responder preguntas o resolver tareas apoyándose en documentos y datos del sistema.
7. Documentar detalladamente el proceso de desarrollo, las decisiones técnicas adoptadas y los resultados obtenidos.

Partes del trabajo

1. Fundamentación Tecnológica y Estado del Arte

- Introducción a los LLMs y análisis de sus limitaciones prácticas (alucinaciones, contexto limitado y dependencia de datos externos).
- Estudio de la arquitectura RAG para enriquecer las respuestas mediante recuperación vectorial.
- Descripción del estándar Model Context Protocol (MCP) y su papel en la agencia y ejecución de acciones en sistemas reales.
- Justificación del stack tecnológico: uso de Quarkus, LangChain4j, bases vectoriales y modelos de embeddings.

2. Análisis y Diseño de la Arquitectura

- Diseño general del sistema, incluyendo diagramas de componentes y flujos de interacción.
- Modelado del pipeline de ingestión y almacenamiento vectorial (estructura de chunks y embeddings).
- Definición del canal de comunicación con el asistente mediante WebSockets.
- Diseño de las herramientas MCP y de su integración con la lógica de dominio del backend.
- Consideraciones de seguridad, aislamiento de herramientas y restricciones del agente.



3. Implementación del Sistema Propuesto

- Configuración del proyecto Quarkus y de los servicios necesarios para operar con LLMs.
- Implementación del módulo de ingestión y procesamiento de documentos (extracción, chunking, embeddings e indexación).
- Desarrollo del asistente inteligente con flujo RAG, gestión de contexto y comunicación en tiempo real.
- Programación y exposición de Tools MCP que permitan al modelo ejecutar acciones sobre el sistema.
- Integración completa de los módulos y validación del flujo extremo a extremo.

4. Experimentación y Resultados

- Evaluación de precisión y utilidad del sistema RAG utilizando distintos tipos de documentos.
- Medición del rendimiento del backend (latencia, tiempos de respuesta, uso de memoria).
- Análisis del comportamiento del agente al ejecutar herramientas MCP.
- Discusión de limitaciones encontradas y comparativa con alternativas existentes.

5. Conclusiones y Líneas Futuras

- Resumen de los principales logros técnicos y aprendizajes.
- Evaluación de la viabilidad del sistema en entornos reales o productivos.
- Propuestas de extensión: soporte multimodal, mejora del RAG, orquestación de múltiples agentes, despliegue en entornos Chrome u offline.

Fases de desarrollo / Planificación temporal

Fase 1: Definición Arquitectónica y Selección del Stack (2 semanas)

- Selección definitiva de los componentes principales: base de datos vectorial, modelo de embeddings y proveedor LLM.
- Diseño de la arquitectura utilizando el modelo C4 y definición de requisitos funcionales y no funcionales (rendimiento, escalabilidad y seguridad).
- Configuración del repositorio, entorno de desarrollo y pipeline básico de CI/CD.

Justificación: Garantizar una base sólida sobre la que construir el sistema, asegurando compatibilidad entre componentes y una arquitectura coherente.

Criterio de éxito: Arquitectura documentada y entorno de desarrollo completamente operativo.

Fase 2: Núcleo del Sistema y Obtención de Datos (3 semanas)

- Inicialización del proyecto Quarkus y configuración de las dependencias necesarias.
- Implementación del pipeline de ingestión de documentos: lectura (PDF, Word, etc.), segmentación en chunks y generación de embeddings.
- Almacenamiento de embeddings en la base vectorial y validación de la recuperación semántica básica.

Justificación: La ingesta y representación vectorial de la información son fundamentales para habilitar búsquedas contextuales mediante RAG.

Criterio de éxito: Recuperación semántica fiable sobre documentos de prueba según umbral mínimo definido.



Fase 3: Lógica del Agente y Orquestación (3 semanas)

- Integración de LangChain4j y conexión con el modelo LLM seleccionado.
- Implementación de la lógica RAG: prompts del sistema, inyección de contexto y flujo de conversación.
- Desarrollo del servidor WebSocket para habilitar comunicación en tiempo real con el asistente.

Justificación: Esta fase dota al asistente de la capacidad de responder consultas de manera contextualizada y dinámica.

Criterio de éxito: Respuestas consistentes en las pruebas, con latencia aceptable (< 500 ms).

Fase 4: Implementación de Capacidades de Agencia mediante MCP (3 semanas)

- Desarrollo de herramientas (tools) específicas según el dominio del proyecto.
- Configuración del enrutamiento de intenciones del modelo y ejecución de acciones dentro de la aplicación.
- Validación del comportamiento autónomo del agente mediante los flujos definidos.

Justificación: MCP permite ampliar el asistente más allá de la consulta de información, habilitándolo para realizar acciones reales en el sistema.

Criterio de éxito: Ejecución correcta de al menos X herramientas definidas y validación completa del flujo de acciones.

Fase 5: Validación, Optimización y Documentación (3 semanas)

- Ajuste de hiperparámetros del LLM y del pipeline RAG (temperatura, tamaño de chunk, top-k, etc.).
- Evaluación del rendimiento, precisión y estabilidad del sistema bajo diferentes escenarios de carga y tipos de documentos.
- Redacción completa de la memoria, anexos técnicos, diagramas arquitectónicos y preparación de la defensa.

Justificación: Asegurar que el sistema final cumple los requisitos del proyecto y que la documentación refleja de forma rigurosa el trabajo realizado.

Criterio de éxito: Memoria final entregada, métricas documentadas y sistema estable en pruebas finales.